

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. :52	Name:ROSHAN YADAV
Class :SYIT	Batch : 1
Date of Assignment :22/07/2026	Date/Time of Submission :22/07/2026

Binary Search Tree: Write a program to:

- a. Create a binary search tree.

code

```
class treeNode:
```

```
    def __init__(self,data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = treeNode('R')
```

```
nodeA = treeNode('A')
```

```
nodeB = treeNode('B')
```

```
nodeC = treeNode('C')
```

```
nodeD = treeNode('D')
```

```
nodeE = treeNode('E')
```

```
nodeF = treeNode('F')
```

```
nodeG = treeNode('G')
```

```
root.left = nodeA
```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
```

```
#nodeC.left = nodeG
```

```
print("root.right.left.data:",root.left.right.data)
```

output

```
===== RESTART: C:/Users/itlab  
root.right.left.data: E  
===== RESTART: C:/Users/itlab  
root.right.left.data: D
```

- b. Insert nodes into a binary search tree.

code

```
class treeNode:
```

```
    def __init__(self,data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = treeNode('R')
```

```
nodeA = treeNode('A')
```

```
nodeB = treeNode('B')
```

```
nodeC = treeNode('C')
```

```
nodeD = treeNode('D')
```

```
nodeE = treeNode('E')
```

```
nodeF = treeNode('F')
```

```
nodeG = treeNode('G')
```

```
nodeH = treeNode('H') #node
```

```
root.left = nodeA
```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
nodeF.right = nodeH #insert node
```

```
print("root.right.right.right.data:",root.right.right.right.data)
```

output

```
===== RESTART: C:/Users/itlab/Desktop
root.right.right.right.data: H
```

- c. Search for a node in a binary search tree.

code

```
class treeNode:
```

```
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right = None
```

```
root = treeNode('R')
nodeA = treeNode('A')
nodeB = treeNode('B')
nodeC = treeNode('C')
nodeD = treeNode('D')
nodeE = treeNode('E')
nodeF = treeNode('F')
nodeG = treeNode('G')
nodeH = treeNode('H') #node

root.left = nodeA
root.right = nodeB
```

```
nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
nodeF.right = nodeH #insert node

print("root.right.right.right.data:",root.right.right.right.data)
#
def search(root,key):
    if root is None:
        return False

    print("checking:",root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)

key = input("Enter element to search: ")

# Search
if search(root, key):
    print("Element Found")
```

else:

print("Element Not Found")

output

```
===== RESTART: C:/Users/itlab/Desktop/roshta  
root.right.right.right.data: H  
Enter element to search: G  
checking: R  
checking: A  
checking: C  
checking: D  
checking: B  
checking: E  
checking: F  
checking: G  
Element Found  
  
===== RESTART: C:/Users/itlab/Desktop/roshta  
root.right.right.right.data:  
H  
Enter element to search: H  
checking: R  
checking: A  
checking: C  
checking: D  
checking: B  
checking: E  
checking: F  
checking: G  
checking: H  
Element Found
```